Golden Bike Shop Integrated System

Compiled Team Decision Records and AI Use Records

Development Team: Kickstarters

Weeks 2 through 8

This document compiles the Team Decision Records and AI Use Records from each weekly design component, in the order the components were completed.

Week 2 – Project Charter

Week ending Sept. 7, 2026

Team Decision Record

Project: The Golden Bike Shop

Team: Kickstarters

Week Ending: Sept 7, 2026

Deliverable: Project Charter

Indicate whether any significant disagreements occurred during this reporting period.

- ☐ No significant disagreements occurred.
- X One or more disagreements occurred.

Decision on the issue – team discussion on which process method to use was discussed. Some back and forth discussion on which method to use and which was best for the purpose of the project.

Competing Positions

Position A: Tinsley stated that Agile would be the preferred method for this project.

Position B: Angela stated that she initially thought of Waterfall for the process to deliver all items at once because she thought it was a new application being built and not one that was going to be integrated.

Resolution: Multiple team members weighed in and agreed that they thought Agile would help deliver items in increments and assist with developing the application at a better rate of speed.

Future Revisit?

- ☐ No
- X Yes

If yes, explain what should be revisited and under what circumstances:

If the requirements change significantly by the customer then this decision may need to be revisited.

AI Use Record

The team used AI to assist with development of the Project Charter. The prompt used provided the required Brief from the Course Guide and requested further information on e-bike business and suggestions regarding ideas for the Golden Bike Shop business context, project constraints, stakeholder information, and to process alternatives being considered.

The AI-generated suggestions were reviewed and the output was modified to better match the Course Guide, removed unnecessary scope and removed duplicated information, adjusted stakeholder roles, schedule assumptions, risks, and scope based on our own decisions.

AI Use Owner: Angela to assist with ideas to give the Charter more context.

Week 3 – Software Functional and Non-Functional Requirements

Week ending Sept. 15, 2026

Team Decision Record

Project: The Golden Bike Shop

Team: Kickstarters

Week Ending: Sept. 15, 2026

Deliverable: Software Functional and Non-Functional Requirements

Disagreements

- ☐ No significant disagreements occurred.
- ☒ One or more disagreements occurred.

Decision 1 — Selection and Refinement of Non-Functional Requirements

Decision on the issue

We needed to determine which non-functional requirements (NFRs) from the AI-generated suggestions should be included in the System Requirements document and how much each requirement needed to be refined before acceptance.

Competing Positions

Position A:

Incorporate the AI-generated NFRs because they address details of the Golden Bike Shop system, including performance, availability, security, auditability, and scalability.

Position B:

Do not accept an NFR simply because it sounds technically appropriate. Each requirement should be specific enough that the development and testing teams can determine objectively whether it has been satisfied. Requirements containing vague terms should be refined or rejected.

Resolution

We chose A and B, using one unaltered A.I. requirement while dissecting and changing the other suggested A.I. generated requirements. We agreed that an NFR should be evaluated based on whether it is relevant to the project, measurable, implementable, and testable. Requirements were retained that could be given objective Acceptance Criteria. Requirements that contained vague or difficult-to-measure language were either refined into smaller requirements or not selected in their original form.

The team also agreed to use a divide-and-conquer approach when a requirement combines multiple expectations. Larger requirements can be separated into smaller requirements that can be implemented and tested independently while still supporting the original business need.

What we gave up

By refining or rejecting broad requirements, the team may have lost some flexibility to address unspecified future scenarios. However, the team determined that clearly defined requirements provide greater value for this project because they give developers and testers a common understanding of what must be implemented and how it will be evaluated.

Future Revisit?

No.

If yes, explain what should be revisited and under what circumstances:

We don't see a need to revisit the requirements that were not included.

Decision 2 — Role-Based Access Control

Decision on the issue

Discussion involving whether role-based access control should be treated as a functional requirement or a non-functional requirement and what the security requirement should actually restrict.

Competing Positions

Position A:

Define access control primarily as preventing unauthorized users from entering or accessing the application.

Position B:

Allow authorized users, including customers and regular employees, to access the application while restricting functions and information according to their assigned roles.

Resolution

Ultimately, the team chose to stick with position B. The application is intended to support multiple types of users, so security should not exclude preventing all users without manager privileges from accessing the system. Instead, the system shall enforce role-based permissions so that users can access only the information and functions authorized for their assigned role. The requirement will be evaluated by testing each defined user role against both permitted and restricted functions.

What we gave up

The team chose not to define security solely as preventing users from entering the application. This provides more flexibility for customers and regular employees while requiring more detailed definition and testing of role permissions.

Future Revisit?

No

If yes, explain what should be revisited and under what circumstances:

This decision is not likely to be brought up again.

Decision 3 — Scalability Requirement

Decision on the issue

The team evaluated the AI-generated requirement:

"Able to handle the current fleet (~400 bikes) and four-store footprint with headroom for modest growth (e.g., additional bikes or a fifth location) without redesign."

We determined that the overall goal was appropriate but that the wording was too broad to test consistently.

Competing Positions

Position A:

Keep the scalability requirement as written because the system should be capable of supporting future growth.

Position B:

Break the requirement into smaller, measurable requirements because "modest growth" and "without redesign" do not define what the system must actually support.

Resolution

We chose Position B and agreed that supporting the current four-store, approximately 400-bike operation is clearly within project scope, but future growth must be defined more specifically before it can become a measurable requirement. Where appropriate, the broader scalability concept will be divided into smaller requirements describing specific numbers of stores, bikes, or other system resources.

What we gave up

We gave up the simplicity of having one broad scalability statement. However, the resulting requirements will be easier to implement, test, and validate because the expected level of growth can be clearly defined.

Future Revisit?

No

If yes, explain what should be revisited and under what circumstances:

This seems like a solid requirement at this time.

Decision 4 — Alignment With the Week 2 Project Charter

Decision on the issue

Decision was needed to determine whether the System Requirements identified during Week 3 required changes to the Week 2 Project Charter.

Competing Positions

Position A:

Update the Project Charter to reflect the additional requirements identified during requirements analysis regarding payments.

Position B:

Keep the Project Charter unchanged unless a requirement falls outside the scope, stakeholders, assumptions, constraints, or objectives already established in the Charter.

Resolution

The team chose Position A. The requirements were reviewed against the Week 2 Project Charter to determine whether they fit the previously established project scope. The team also identified the importance of ensuring that requirements do not conflict with the Charter and updated the Project Charter to maintain consistency.

What we gave up

We gave up having a full, clear picture of everything in advance. However, with us planning on an agile methodology, revisiting our expectations is part of the process.

Future Revisit?

No. Project Charter revised as recommended.

A.I. Use Record

- **The component -** Week 3 Software Functional & Nonfunctional Requirements
- **Which AI Tool was used –** Chat GPT-5.6 Luna
- **What was asked:** The Golden Bike Shop brief was provided at the top. Then the entire Project Charter submitted from last week. Then asked it to: "Please write 4 Non-Functional requirements for this project. Look past performance and provide 4 or 5 non-functional requirements that pertain to availability, security, usability, maintainability or portability."
- **What it returned:** A summary of the AI-generated response. It returned 4 Non-Functional Requirements and gave the category under which section they fell into.

NFR-01 — Availability

  - The system shall be available to authorized users at least 99.5% of the time during normal business operating hours, excluding scheduled maintenance that is communicated to users in advance.
  - Category: Availability

NFR-02 — Security and Access Control

- The system shall enforce role-based access control so that users can access only the functions and information appropriate to their assigned role, with manager-level functions restricted to authorized managers and administrators.
- Category: Security

NFR-03 — Usability

- The system shall allow a trained store, rental, or service employee to locate the availability and current status of a specific inventory item or rental bicycle within three user interactions or fewer from the main system interface.
- Category: Usability

NFR-04 — Maintainability

- The system shall use a modular architecture that allows changes to one operational area (retail inventory, rental fleet, telemetry, or service scheduling) to be made without requiring changes to unrelated operational modules.
- Category: Maintainability
- What we changed and why.
  - I added one of the AI Non-Functional requirements that I thought would work with the Golden Bike Shop, which was the Security and Access Control requirement, keeping the verbiage as is because it seemed appropriate to the project and it seemed specific enough without being too vague.
  - While reviewing the wording for the AI provided usability requirement, it seemed like the wording of it could become subjective. For example, "what counts as an interaction? Would selecting a drop-down count? Typing search term? Etc. The same decision process was used for reviewing the AI provided Maintainability requirement, as the words "modular architecture" seems like it is meaningful, however, we have not reached the architectural design phase to know what to test for in this requirement.

- A simpler approach was used to change the wording the Maintainability and Usability requirements entirely to make sure that the requirements could be tested without assuming the architectural design before it has been generated.
- Acceptance Criteria was added to each of the requirements while considering how to test and verify the listed requirement.
- Another requirement not generated by AI was also added to the Non-Functional requirements that would fit the purpose of portability, ensuring the system shall operate correctly on the current versions of the organization-approved web browsers.

AI Use Owner – Angela Colton

Week 4 – User Interface Designs

Week ending Sept. 20, 2026

Team Decision Record

Project: The Golden Bike Shop

Team: Kickstarters

Week Ending: Sept. 20, 2026

Deliverable: User Interface Designs

Disagreements

- ☐ No significant disagreements occurred.
- ☒ One or more disagreements occurred.

Decision 1 — Deciding Which Screen to Use for the Discussion Post

Decision on the issue

We needed to determine which screen should be used for the Week 4 Discussion Post and which journey it should represent. We debated whether to use a customer-facing screen ("Check my bike service status") or an employee-facing inventory lookup screen tied to REQ-001.

Competing Positions

Position A: Use the customer-facing service status screen because it is simple, easy to sketch, and directly understandable to users.

Position B: Use the employee-facing inventory lookup screen because it supports the highest-priority requirement (REQ-001), is central to Golden Bike Shop's business, and aligns better with the rubric's requirement to demonstrate core system functionality.

Resolution

We chose Position B — the employee inventory lookup screen.

This decision aligned with the rubric, the business priorities, and the screens already drafted. We came to an agreement that the Discussion Post should highlight the most critical workflow: employees looking up inventory across all four store locations.

What we gave up

By choosing the employee-facing screen, we gave up:

- A simpler customer-facing screen
- A potentially easier sketching process
- A lower-complexity UI example

However, we determined that demonstrating the highest-priority requirement provided greater value and clarity for both stakeholders and grading.

Future Revisit?

No.

We do not anticipate revisiting this decision because REQ-001 is the top priority and the chosen screen best represents the system's core functionality.

Decision 2 — Deciding How Many Screens to Build

Decision on the issue

We needed to determine whether the assignment required 3–4 screens total or 3–4 screens per journey, as it appeared that the rubric wording was unclear to us and Angela completed way too many screens during the process of writing out the assignment.

Competing Positions

Position A: Create 3–4 screens total, keeping the submission small and simple.

Position B: Create 3–4 screens per journey, because each journey must follow a user "all the way through" a workflow.

Resolution

We chose Position B — 3–4 screens per journey.

The professor clarified during office hours that Week 4 does not build on previous weeks and that each journey must be fully represented with its own screens.

What we gave up

- A smaller, simpler submission
- A single consolidated UI
- Less work overall

But the team determined that fully representing each journey was necessary to meet rubric expectations.

Future Revisit?

No. The rubric and professor clarification make this decision final.

Decision 3 — Selecting the Three User Journeys

Decision on the issue

We needed to choose which three user journeys best represent the Golden Bike Shop system and align with the screens already drafted.

Competing Positions

Position A: Use customer-facing journeys (e.g., service status, online ordering).

Position B: Use employee-facing journeys tied to the highest-priority requirements:

- Retail Manager
- Rental Employee
- Lead Service Staff

Resolution

We chose Position B — the three employee-facing journeys.

These journeys map directly to the highest-priority requirements and represent the core business domains: retail, rental, and service.

What we gave up

Customer-facing journeys and Lower-priority rental manager history screens

Future Revisit?

No. These journeys align with the requirements and the screens already drafted.

Decision 4 — Incorporating AI-Generated Alternative Designs

Decision on the issue

We needed to determine which AI-generated UI enhancements should be incorporated into our final designs.

Competing Positions

Position A: Adopt the AI-generated designs fully, including icons, dashboards, maps, and detailed layouts.

Position B: Keep our original simplified designs but incorporate only the AI enhancements that improve usability and align with requirements.

Resolution

We chose Position B — keep our original designs and selectively incorporate AI improvements. Specifically, we adopted:

- Inventory Date
- Filter options
- Navigation menu
- Out-of-Stock indicators

We rejected:

- Large icons for every bike
- Split-screen layouts
- Map-based dashboards
- Overly detailed main screens

What we gave up

- More visually complex screens
- Potentially richer dashboards
- Novel UI concepts (e.g., map-based bike tracking)

But the team determined that simplicity, clarity, and alignment with requirements were more important.

Future Revisit?

Yes — only if future requirements introduce:

- GPS tracking
- Visual bike identification

A.I. Use Record

- **The Component:-** Week 4 - User Interface Designs
- **Which AI Tool was used –** Microsoft Copilot

What was asked: The Golden Bike Shop brief was provided at the top, then Week 3 requirements document and the constraints were provided, and the draft sketched screens. The AI was then asked:

“Please provide two or three alternative approaches to the same screen. The alternatives must follow the requirements and constraints.”

This request was repeated for all three journeys:

1. The Retail Manager Inventory Lookup Screen

2. The Rental Bike Status Update Screen

3. The Service Bay Booking Screen

The goal was to compare our team’s original UI sketches with alternative UI layouts generated by AI.

What it returned:

Alternative Set 1 - Retail Manager Inventory Lookup Screen

Alternative A — Dashboard Layout

Inventory Date field

Filters above results table

Top navigation menu (Dashboard, Inventory, Orders)

“Out of Stock” dashboard section highlighted in red

Sidebar menu for quick navigation

Alternative B — Guided Search Workflow

Step 1: Login

Step 2: Search by UPC or Name

Step 3: Results summary

Step 4: Detailed inventory breakdown

Alternative C — Split-Screen Interface

Left: Search + filters

Right: Dynamic results with store inventory and aisle map thumbnails

Alternative Set 2 - Rental Bike Status Update Screen

Alternative A — Two-Panel “Browse + Details” Layout

Left: Bike list + filters

Right: Detailed bike info (status, battery, range, lock status)

“Rent Bike” button only enabled when status = Available

Alternative B — Guided Rental Workflow

Step 1: Select bike type

Step 2: Select available bike

Step 3: Review telemetry + confirm rental

Alternative C — Card-Based Quick Action Layout

Each bike shown as a card with status badge

Quick actions: Rent, Update Status, View Telemetry

Alternative Set 3 - Service Bay Booking Screen

Alternative A — Horizontal Booking Layout

Appointments displayed horizontally instead of vertically

Included unrelated Tesla imagery (AI error)

Alternative B — Service Details Screen

Included serial numbers

Displayed servicing details instead of scheduling

Suggested adding serial numbers to our “View Details” modal

What we changed and why.

Retail Manager Screen

We kept our original simplified design but added: Inventory Date, Filters, Navigation menu, Out-of-Stock indicators

We rejected large icons, split-screen layout, overly detailed main screen,

Reason: These additions improved usability without overwhelming the user.

Rental Bike Status Update Screen

We kept our original single-screen layout for speed and simplicity.

We rejected map-based dashboard (too complex, not required), and large bike icons (would require uploading images for every bike).

We accepted the idea of a two-panel layout and guided workflow, but did not implement them due to complexity.

Service Bay Booking Screen

We kept our original calendar + daily bay view.

We rejected horizontal Tesla-based layout (irrelevant to Golden Bike Shop).

We accepted: Adding serial numbers to the “View Details” modal window.

Why We Made These Changes

The AI alternatives revealed assumptions we had made about simplicity. Some AI features improved usability (filters, inventory date, navigation). Some AI features were too complex or irrelevant (maps, Tesla imagery).

We prioritized:

Clarity

Alignment with requirements

Avoiding cognitive overload

Keeping screens testable and implementable

Future Revisit?

No. The team does not anticipate revisiting the rejected AI alternatives unless future requirements introduce:

GPS tracking

Visual bike identification

Manager dashboards

Expanded service details

AI Use Owner – Angela Colton

Week 5 – Application Architecture

Week ending Sept. 27, 2026

Team Decision Record

Project: The Golden Bike Shop

Team: Kickstarters

Week Ending: Sept. 27, 2026

Deliverable: Application Architecture

Disagreements

- ☐ No significant disagreements occurred.
- ☒ One or more disagreements occurred.

Decision 1 - Selection of Architecture

Decision on the issue

The primary decision for the week was what overall architecture to use for our system.

Competing Positions

Position A:

Use N-tier architecture. One tier for the database, one for the backend logic applications, and one for the UI application.

Position B:

Use backend-for-frontend architecture. One frontend for the customer side, one frontend for the internal side, each with their own backend application, pulling from one or more databases.

Resolution

We chose Position A. Our existing requirements read as being built around a unified system, with no hard split for any particular roles. We factored in the scale of the bike shop business and the expectation we decided upon in an earlier week that any growth would be relatively small, and decided that a simpler architecture was the right call.

What we gave up

Flexibility and scalability of the system.

Future Revisit?

No.

If yes, explain what should be revisited and under what circumstances:

We would only revisit this if the company came back and told us they expect massive growth.

Decision 2 – Databases vs tables

Decision on the issue

Once we had decided on n-tier, we needed to determine how to house the data for the system.

Competing positions

Position A:

Separate data into its own databases.

Position B:

Place all data within a single database, organized by tables.

Resolution

We chose Position B. The primary reason for this was the extent to which our data is linked. Many if not most of our tables would require a foreign key, and some data needs to be joinable. The lines between which data could live separately and which needed to be housed in the same database were not clear or clean.

What we gave up

The ability to have separate database schema, or for databases to survive disasters separately.

Future Revisit?

No

Decision 3 – Number of entities in the business logic tier

Decision on the issue

Once we had decided on n-tier, we needed to determine to what extent we would be splitting out the functionality in the business logic tier. This built on top of decision 1 and 2, but was its own discussion and decision.

Competing Positions

Position A:

Use a monolithic structure for our business logic application. The application can be set up with logical services, but is deployed and maintained as a single application.

Position B:

Modularization. Split the backend based on the domains within the business. One application for retail, one for rentals, one for service work.

Resolution

We chose Position A. There is enough overlap between the domains that they are expected to have to share some functionality. Additionally, building things as multiple applications requiring separate deployments and maintenance would have added significant complexity, and likely require the frontend application to know about the separate backends. This was also influenced by our decision to have a unified database. Splitting functionality into separate apps that all require access to the same database adds additional complexity around data synchronization.

What we gave up

The ability to maintain domains separately, or for domains to survive disasters separately.

Future Revisit?

No

Decision 4 – SLA for NFR-001

Decision on the issue

One of our non-functional requirements had an SLA of one minute for any inventory changes to be reflected within the database.

Competing Positions

Position A:

Keep NFR-001 as is. It was built to support asynchronous operations and allow some lag within the system, especially during times of high throughput.

Position B:

Change NFR-001 to a much lower value. This requires us to commit to synchronous operations when inventory needs to change within the system, but in exchange makes the system atomic, so we should never be able to retrieve stale data.

Resolution

We chose Position B. The way we had created our architecture seemed to be leaning toward synchronous operations, and the positive of having accurate counts at all times outweighs any negatives.

What we gave up

No delay on posting an inventory change.

Future Revisit?

No

A.I. Use Record

- **The Component:-** Week 5 – Application Architecture Design
- **Which AI Tool was used –** Microsoft Copilot

What was asked: Week 3 requirements document and the Week 4 UI document were provided. The AI was then asked:

"We selected N-tier architecture because both the Requirements Document and the UI Design Document already assume a single unified front-end, a single shared application layer, and a single unified database, with role-based access controlling what each user sees. I need the strongest case against that choice. Given the requirements, constraints, and reasoning. I need for you to attack the decision to use N-tier architecture, not replace it."

The goal was to compare our team's original UI sketches with alternative UI layouts generated by AI.

What it returned:

The Strongest Case Against Choosing N-Tier Architecture

Even though N-tier is simple and familiar, it introduces several serious risks and limitations when applied to the Golden Bike Shop system as described in your Requirements and UI Design documents. Below are the most defensible arguments against the decision.

1. N-tier creates a single point of failure for the entire business

Your system supports:

- Multi-location inventory

- Real-time rental status
- Service bay scheduling
- Telemetry updates
- Historical sales

All of these depend on one backend and one database.

If the backend goes down:

- All four stores lose access
- Rental operations stop
- Service bay scheduling stops
- Inventory lookup stops
- Telemetry ingestion stops

This means the entire business is dependent on one server, which is a fragile design for a system with multiple physical locations and real-time operational needs.

2. N-tier forces unrelated domains into one backend, increasing coupling

Your system has four distinct domains:

- Retail inventory
- Rental fleet
- Service scheduling
- Telemetry

Each domain:

- evolves independently
- has different performance needs
- has different update frequencies
- has different operational workflows

N-tier collapses all of these into one application layer, which causes:

- tight coupling between unrelated features
- higher regression risk
- more complex deployments
- harder testing
- slower development over time

This makes the system harder to maintain as it grows.

3. N-tier limits scalability and future growth

Your requirements include:

- real-time telemetry
- multi-location inventory
- rental updates
- historical reporting

These features have different scaling profiles.

Telemetry may spike. Inventory queries may be heavy. Service scheduling may require fast conflict detection. Sales reporting may require large queries.

N-tier forces all traffic through one backend, which:

- cannot scale individual domains independently
- cannot isolate high-load components
- cannot prioritize critical operations
- requires vertical scaling (bigger server), not horizontal scaling (more servers) This limits future growth and flexibility.

4. N-tier increases long-term technical debt

Because all business logic lives in one backend:

- every new feature increases backend complexity
- every change risks breaking unrelated features
- testing becomes harder
- deployments become riskier
- refactoring becomes more expensive
- onboarding new developers becomes slower

Over time, the backend becomes a monolithic ball of mud, even if it starts clean. This is a predictable long-term consequence of the N-tier decision.

5. N-tier does not reflect the natural structure of the business

Golden Bike Shop has clear domain boundaries:

- Inventory
- Rental
- Service
- Telemetry
- Sales

These domains are independent in the real world, but N-tier forces them into:

- one front-end
- one backend
- one database

This means the architecture does not mirror the business structure, which makes the system harder to evolve as the business changes.

6. N-tier makes quality attribute trade-offs worse

By choosing N-tier, the team traded away:

- modifiability
- scalability
- reliability
- performance
- maintainability

in favor of:

- simplicity
- familiarity
- ease of initial implementation

This is a short-term optimization that creates long-term risk.

What we changed and why.

After reviewing the AI-generated objections, we determined that only one part of the strongest objection required action: the risk that an N-tier architecture creates a single point of failure because all business logic runs in one backend. The AI argued that this could make the system fragile under load and vulnerable to outages across all four store locations.

We evaluated this objection against our actual requirements and constraints. The Golden Bike Shop system does contain multiple domains (inventory, rentals, service scheduling, telemetry), and the backend does coordinate all of them. However, the business operates only four stores, traffic volume is modest, and telemetry data is simple. Because of this, the objection is theoretically strong but practically weak for our system.

The team therefore did not change the architecture. The N-tier decision still stands because it aligns with the Requirements Document and UI Design Document, both of which assume a unified front-end, unified backend, and unified database.

Instead, we added one operational safeguard: the backend should be deployed with redundancy (e.g., multiple instances behind a load balancer) and the database should have automated backups and failover. This addresses the single-point-of-failure concern without altering the architecture itself.

Why We Made These Changes

We made these changes because the AI's strongest objection identified a real risk that centralizing all business logic in one backend does create a potential bottleneck and single point of failure. Even though the Golden Bike Shop's scale makes this risk small, it is still meaningful enough to warrant mitigation.

However, the objection did not justify replacing or restructuring the architecture. The requirements and UI design both assume a unified system with shared data and shared business logic. We also agreed earlier that expected growth would be modest. Because of this, the benefits of N-tier (simplicity, clarity, ease of deployment) outweigh the theoretical scalability and coupling concerns raised by the AI.

The only part of the objection that "lands" for this system is the operational fragility of a single backend instance. Adding redundancy and failover addresses that concern directly while preserving the architecture that best fits the project's scope, requirements, and constraints.

Future Revisit?

No. The team does not anticipate revisiting this decision because the objection raised by the AI. While theoretically strong, it does not materially impact the Golden Bike Shop system as scoped.

AI Use Owner – Angela Colton

Week 6 – Data Design

Week ending Oct. 4, 2026

Team Decision Record

Project: The Golden Bike Shop

Team: Kickstarters

Week Ending: October 4, 2026

Deliverable: Data Design Document

Disagreements

- ☐ No significant disagreements occurred.
- ☒ One or more disagreements occurred.

Decision 1 — Determining the Number of Environments

Decision on the issue

The team needed to determine how many environments were appropriate for supporting deployment and testing of the Golden Bike Shop system.

Competing Positions

Position A:

Angela suggested the use of three environments: Development, QA, and Production, with QA serving as both the testing and staging environment.

Position B:

Joseph discussed adding additional test environments such as UAT1 and UAT2 to allow multiple testing efforts to occur simultaneously and more closely match larger enterprise practices.

Resolution: We chose Position A and agreed that Development, QA, and Production were sufficient for a four-store operation. QA would function as both the testing and staging environment, while maintaining its own non-production database.

What we gave up

- Concurrent testing across multiple dedicated environments
- Additional separation between testing activities
- Enterprise-style deployment practices used by larger organizations

We determined that the added complexity and maintenance effort were not justified for a business of this size.

Future Revisit?

Yes. If the company expands significantly beyond four stores or development teams begin working on multiple major features simultaneously, additional testing environments may become necessary.

Decision 2 — Choosing How to Handle Rental Pricing

Decision on the issue

Aela dentified that rental pricing was not represented in the original data model and needed to determine whether pricing should be stored directly on rental bikes or separated into its own entity.

Competing Positions

Position A:

Store the rental rate directly on each RentalBike record.

Position B:

Create a separate RentalPricing entity that manages rental rates independently from specific bike records.

Resolution: Angela and Aela chose Position B and added a RentalPricing entity to the data model.

The new entity supports maintaining rental rates independently from rental bike records and allows the system to accommodate future pricing changes more easily.

What we gave up

- A simpler data model
- Fewer entities and relationships
- Slightly reduced implementation complexity

However, we gained improved support for pricing changes and historical rate management.

Future Revisit?

No. We believe rental pricing is a fundamental business requirement that should remain part of the design.

Decision 3 — Selecting Edge Cases for Data Model Validation

Decision on the issue

The team needed to decide whether to primarily use the edge cases provided by the assignment instructions or develop edge cases based on the actual requirements and workflows represented in the Golden Bike Shop system.

Competing Positions

Position A:

Focus primarily on the example edge cases listed in the assignment instructions.

Position B:

Develop additional edge cases based on the team's requirements, architecture, and business workflows.

Resolution: We chose Position B.

The team used the assignment examples as a starting point but expanded the list to include conditions such as concurrent rentals, overlapping service appointments, telemetry outages, inventory conflicts, late returns, and bike availability issues.

What we gave up

- A shorter and simpler validation exercise

- Reliance solely on instructor-provided examples

Instead, the team created scenarios that more accurately reflected how the Golden Bike Shop system would operate across four store locations.

Future Revisit?

Only if new functionality or business processes are introduced.

Decision 4 — Determining Whether Reservations Should Support Multiple Bikes

Decision on the issue

The team needed to determine whether a single reservation should allow customers to reserve multiple rental bikes at the same time or whether reservations should be limited to a single bike.

Competing Positions

Position A:

Allow multiple bikes per reservation by introducing a ReservationBikeAssignment bridge entity. This would create a many-to-many relationship between reservations and rental bikes and allow customers to reserve several bikes within a single reservation transaction.

Position B:

Limit each reservation to a single rental bike. Customers who need multiple bikes would create multiple reservations rather than using a single reservation containing several bikes.

Resolution: We chose Position B and designed the Reservation entity so that each reservation is associated with only one rental bike.

Although the lecture example discussed reservations containing multiple bikes, our team intentionally constrained reservations to a single bike based on the requirements, user interface designs, and business workflows documented in Weeks 4 and 5. Customers requiring multiple bikes create multiple reservations instead. As a result, a ReservationBikeAssignment bridge entity was not included in the final data model.

This decision kept the reservation process consistent with our previously approved project artifacts and reduced complexity within the data model.

What we gave up

- Group reservations involving multiple bikes
- Family reservations managed through a single transaction
- A more flexible reservation structure that could support packages or bundled rentals By choosing a one-bike-per-reservation model, we simplified reservation creation, reservation management, availability checking, and conflict detection.

Future Revisit?

Yes. If future business requirements include family rentals, group reservations, guided tours, or reservation packages involving multiple bicycles, the team would revisit this decision and likely introduce a ReservationBikeAssignment bridge entity to support a many-to-many relationship between reservations and rental bikes. For the current scope of the Golden Bike Shop system, the added complexity was not justified by the documented requirements.

A.I. Use Record

The Component:- Week 6

Which AI Tool was used – Microsoft Copilot

Technique Use: Recognizing Confident Error

What was asked:

We asked an AI tool to generate and review a data model for the Golden Bike Shop system, including support for customers, rental bikes, rentals, inventory management, sales, and service appointments. We also asked the AI to identify any missing entities, relationships, or edge cases that should be considered in the design.

What it returned:

The AI suggested several modifications to the data model, including the addition of bridge tables to support many-to-many relationships. One recommendation was to create a reservation-to-bike assignment table that would allow a single reservation to contain multiple rental bikes. The AI also suggested adding rental pricing history support and identified several potential edge cases related to reservations, rentals, and inventory management.

What we changed and why:

The AI suggested including a table that allowed multiple bike rentals for a single reservation by introducing a reservation-to-bike assignment entity. After reviewing the recommendation, we decided not to implement this change. Our Week 4 user interface designs and Week 5 architectural documents were based on the assumption that each reservation is associated with a single rental bike. Adding support for multiple bikes per reservation would have required significant changes to the user interface, business rules, architecture documentation, and entity-relationship diagram. Instead, we retained the original design and allowed customers to create multiple reservations if multiple bikes are needed. This approach maintains consistency across all project deliverables while meeting the core reservation requirements identified for the system.

Why We Made These Changes

We made these changes to ensure consistency across all project artifacts and to avoid introducing functionality that had not been identified during requirements gathering. While the AI recommendation was technically valid, it increased the complexity of the data model without providing a clear benefit that justified modifying the existing user interface and architectural designs. This exercise demonstrated that AI-generated solutions can appear complete and correct, but still require human evaluation against project requirements and design constraints.

Future Revisit?

Yes. If future stakeholder feedback indicates a need for family reservations, group rentals, or other scenarios where multiple bikes should be reserved at the same time, the team may revisit this decision. In that case, a reservation-to-bike assignment table could be added to support a many-to-many relationship between reservations and rental bikes. For the current project scope, however, a one-bike-per-reservation model was determined to be sufficient.

AI Use Owner – Angela Colton

A.I. Use Record

The Component:- Week 6

Which AI Tool was used – Microsoft Copilot

What was asked:

Copilot was given the overview sections from our Project Charter and System Requirements documents, and then asked, “As an expert software engineer, please pick one area of the described system for the Golden Bike Shop and generate a data model for it. Please ask any questions upfront, including requests for any specific requirements.”

What it returned:

It asked several clarifying questions. Most were answered, but questions about edge cases were left unanswered to keep its response honest. It also stated that it was leaning toward the rental domain, and I told it to go with that option.

With the clarifications, it returned:

🧭 Core Takeaway

The rental domain revolves around Bike, Rental, Reservation, TelemetrySnapshot, and Operational Status. The model below cleanly separates state, events, and availability logic, while keeping integration points open for sales and service.

📐 Rental Domain Data Model (Relational)

🚲 1. Bike

Represents each physical rental bike. Shares product catalog identity with retail.

- Bike
  - bike_id (PK)
  - product_id (FK → Product)
  - serial_number (unique)
  - store_id (FK → Store) — current physical home location
  - is_ebike (boolean)
  - battery_health_score (numeric 0–1 or 0–100)
  - status (FK → BikeStatus)
  - created_at
  - updated_at

Notes:

- status is a derived operational state (available, rented, in_service, reserved, offline, etc.).
- Battery health is long-term; current battery % lives in telemetry.

🚦 2. BikeStatus

Enumerates operational states.

- BikeStatus
  - status_code (PK)
  - description

Examples:

- available
- rented
- reserved
- in_service
- offline
- unavailable_other

📍 3. TelemetrySnapshot

Latest telemetry only — no history unless you later decide you want it.

- TelemetrySnapshot
  - bike_id (PK, FK → Bike)
  - latitude
  - longitude
  - battery_percentage
  - last_reported_at — timestamp of telemetry arrival
  - updated_at — timestamp of DB update

Notes:

- PK = bike_id ensures "latest snapshot only."
- If last_reported_at is older than threshold (e.g., 20 minutes), system may set bike status → offline.

🗓️ 4. Reservation

Future bookings. Does not represent active rentals.

- Reservation
  - reservation_id (PK)
  - bike_id (FK → Bike)
  - customer_id (FK → Customer)
  - start_time
  - end_time
  - created_at
  - updated_at
  - status (enum: active, cancelled, expired)

Notes:

- A bike can have multiple future reservations.
- Overlaps are prevented by constraint:
  - EXCLUDE USING gist (bike_id WITH =, tstzrange(start_time, end_time) WITH &&)

🚲 5. Rental

Represents an active or completed rental.

- Rental
  - rental_id (PK)
  - bike_id (FK → Bike)
  - customer_id (FK → Customer)

    - checkout_time
    - expected_return_time
    - actual_return_time (nullable)
    - origin_store_id (FK → Store)
    - return_store_id (FK → Store, nullable)
    - created_at
    - updated_at
    - status (enum: active, completed, late, cancelled)

Notes:

- Late returns automatically affect availability.
- Bikes can be returned to a different store.

🚧 6. ServiceLink (Minimal)

You said you don't need rental→service linkage, only bike→service. So:

- ServiceJob
    - service_job_id (PK)
    - bike_id (FK → Bike)
    - start_time
    - end_time (nullable)
    - status (enum: scheduled, in_progress, completed, no_show)
    - created_at
    - updated_at

Notes:

- When a service job is created or started, the bike's status flips to in_service.

🏪 7. Store

Simple store representation.

- Store
    - store_id (PK)
    - name
    - address
    - timezone
    - created_at
    - updated_at

👤 8. Customer

Unified identity across domains.

- Customer
    - customer_id (PK)
    - first_name
    - last_name
    - email
    - phone

    - created_at
    - updated_at

👮 9. Role & Access Control

Domain-scoped write permissions.

- Role
    - role_id (PK)
    - name Examples:
    - owner
    - store_manager
    - rental_manager
    - rental_staff
    - service_staff
    - service_staff_lead
    - sales_staff
- User
    - user_id (PK)
    - name
    - email
    - role_id (FK → Role)
- RoleDomainPermission
    - role_id (FK → Role)
    - domain (enum: sales, rental, service)
    - can_write (boolean)

🔗 Relationships Overview

- Bike 1—N Rental
- Bike 1—N Reservation
- Bike 1—1 TelemetrySnapshot
- Bike 1—N ServiceJob
- Customer 1—N Rental
- Customer 1—N Reservation
- Store 1—N Bike
- Store 1—N Rental (origin/return)

🧠 Availability Logic (Derived, not stored)

Availability is computed from:

1. Bike.status
2. Active rentals
3. Future reservations
4. Service jobs
5. Telemetry health (offline threshold)

Pseudo-logic:

Code

IF bike.status = in_service THEN unavailable

ELSE IF active rental exists THEN unavailable

ELSE IF future reservation overlaps desired window THEN unavailable

ELSE IF telemetry stale THEN unavailable_with_warning

ELSE available

📊 Leadership's Likely Top 3 Questions (Your "best

judgment" request)

1. How many rental bikes are currently unavailable, and why? (Breakdown: rented, reserved, in service, offline)

2. What is the utilization rate of the rental fleet by store? (Active rentals / total bikes over time)

3. Which e-bikes are at risk due to low battery or poor battery health? (Battery % < threshold OR health score < threshold)

🧩 Why This Model Works Well

- Clean separation of state (BikeStatus), events (Rental, Reservation, ServiceJob), and telemetry.
- Supports real-time availability computation.
- Plays nicely with the sales domain (shared product catalog).
- Integrates with service domain without forcing deep coupling.
- Supports role-based write access cleanly.
- Leaves room for future enhancements (battery degradation modeling, telemetry history, dynamic pricing).

What we changed and why.

The only change we made following this was to add rental pricing to our model. In noticing the AI response was missing it, we realized ours was as well.

Future Revisit?

No. Aside from helping us identify the missing rental pricing component, the generated model did not reveal additional gaps in our design.

AI Use Owner – Aela Mason

Week 7 – Software Quality & Test Plan

Week ending Oct. 11, 2026

Team Decision Record

Project: The Golden Bike Shop

Team: Kickstarters

Week Ending: October 11, 2026

Deliverable: Software Quality and Test Plan

Disagreements

- ☐ No significant disagreements occurred.
- ☒ One or more disagreements occurred.

Decision 1 — Testing Prioritization

Decision on the issue

Determine which areas of the Golden Bike Shop system should receive the highest level of testing given project schedule and resource limitations.

Competing Positions

Position A: Distribute testing effort approximately equally across all modules to ensure broad coverage.

Position B: Allocate the majority of testing effort to functionality with the greatest business risk and customer impact, even if lower-risk features receive less testing.

Resolution

We chose Position B. The team determined that bike availability tracking, reservation validation, rental processing, inventory synchronization, and service scheduling have the greatest potential impact on customer satisfaction and daily business operations. Failures in these areas could result in unavailable rentals, lost sales, scheduling conflicts, and inaccurate business information.

What we gave up

- Reduced testing depth for lower-risk administrative features.
- Less comprehensive coverage of infrequently used employee-management functions.

Future Revisit?

Yes.

Reason:

The team will revisit this decision if usage patterns change, if lower-priority functions begin generating defects, or if new business requirements increase the importance of currently low-risk features.

Decision 2 — Testing Scope for Administrative Functions

Decision on the issue

Determine whether employee account management and profile maintenance functions should receive the same level of testing as customer-facing business operations.

Competing Positions

Position A: Apply the same degree of testing coverage to administrative account setup, role assignment, and profile maintenance as customer-facing functionality.

Position B: Perform limited but targeted testing of administrative functions and focus additional testing effort on customer-facing business processes.

Resolution

We chose Position B. The team concluded that employee account creation, role assignment, and profile updates are relatively infrequent activities involving straightforward business logic. While defects in these areas may create inconvenience for employees, they are less likely to affect customers directly than failures in rental processing, inventory management, or service scheduling.

What we gave up

- Comprehensive testing of all account-management scenarios.
- Extensive validation of employee profile maintenance workflows.

Future Revisit?

Yes.

Reason:

The team will revisit this decision if account creation becomes more frequent, if users begin reporting recurring login or permission problems, or if profile-data errors begin affecting customer service or operational efficiency.

Decision 3 — Scope of Additional Testing Recommendations

Decision on the issue

Determine whether AI-identified testing gaps should be implemented as formal detailed test cases or documented as future testing considerations.

Competing Positions

Position A: Create additional formal test cases for every identified testing gap.

Position B: Document the identified gaps as future testing considerations while maintaining focus on the project's highest-priority requirements.

Resolution

We chose Position B.

Why? The project timeline, team size, and scope limit the number of detailed tests that can be formally documented. The identified gaps were considered valuable but secondary to requirements already covered by the existing test cases.

What we gave up

- Detailed execution procedures for every identified gap.
- Additional verification coverage in lower-priority areas.

Future Revisit?

Yes.

Reason:

If development progresses ahead of schedule or identified risks begin occurring during testing, additional formal test cases may be added.

A.I. Use Record - Identification of Testing Gaps

- **The Component:-** Week 7 – Software Quality and Test Plan
- **Which AI Tool was used –** Microsoft Copilot

What was asked: The Golden Bike Shop documents from Week 3 through Week 6 were provided, as well as the initial draft of the document. The AI was then asked:

"Review the Software Quality & Test Plan and identify any areas where testing coverage may be incomplete. Provide only gaps and not a revised test plan."

What it returned:

The AI identified several potential gaps, including:

- Concurrency testing during rental transactions
- Telemetry failure and degraded-data scenarios
- Negative security testing beyond basic role access validation
- Cross-domain workflow and integration testing
- Enterprise-scale load testing
- Disaster recovery testing

What we changed and why:

The team reviewed each suggestion and evaluated whether it was appropriate for the Golden Bike Shop system and project scope.

Accepted changes:

The recommendations were incorporated into future testing considerations and may be implemented as additional test scenarios if time and project scope permit. The team added consideration for:

- Concurrency testing during rental and reservation processing
- Telemetry outage and degraded-data scenarios
- Expanded unauthorized-access testing
- Cross-domain workflow validation to verify that updates in rentals, inventory, servicing, and telemetry are reflected correctly throughout the system

Rejected changes:

Enterprise-scale load testing was rejected because the business consists of four locations and approximately 400 rental bikes, making large-scale performance testing disproportionate to project needs.

Disaster recovery testing was rejected because it falls outside the scope of this academic project and does not directly address the highest-priority requirements identified by the team.

Why We Made These Changes

The AI review helped identify realistic testing gaps and provided additional evidence that testing decisions were reviewed critically rather than accepted automatically.

Future Revisit?

Yes.

Reason:

The team intends to revisit the accepted AI recommendations if new functionality significantly expands rental processing, telemetry integration, security requirements, or cross-domain workflows. Additional concurrency, telemetry, integration, and security testing may become necessary as implementation progresses. The team does not currently plan to revisit enterprise-scale load testing or disaster recovery testing because those recommendations remain outside the scope of the current project.

AI Use Owner – Angela Colton

A.I. Use Record – Testing Prioritization and Risk Acceptance Analysis

1. The Component: Week 7 Group Discussion Post – Testing Priorities and Testing Limitations

2. Which AI Tool Was Used: Microsoft Copilot

What was asked:

The Golden Bike Shop project requirements, architecture, and testing plan were reviewed. The AI was asked:

"Identify which system functions should receive the highest testing priority and recommend lower-risk areas where limited testing may be acceptable. Explain the business risks associated with those decisions and identify conditions that would require those decisions to be revisited."

What it returned:

The AI identified bike availability tracking, reservation validation, and rental processing as the highest-priority testing areas because failures in those functions could directly affect customers and business operations. It also identified administrative account setup, role assignment, and employee profile maintenance as lower-risk functions that could reasonably receive less testing due to their low frequency of use and lower business impact.

The AI further suggested documenting:

- Risks associated with incorrect bike availability information.
- Risks associated with limited testing of account management functionality.
- Risks associated with limited testing of employee profile updates.
- Conditions that would require the team to revisit those testing decisions in the future.

What we changed and why:

The team reviewed the recommendations and agreed that bike availability tracking represents the highest business risk in the system. We documented why rental processing, reservation validation, and bike availability should receive the greatest testing focus.

We also accepted the recommendation to formally document areas where testing effort would be limited. Administrative account setup, role assignment, and employee profile maintenance were identified as lower-risk functions because they occur infrequently, involve relatively simple business logic, and primarily affect internal employees rather than customers.

The team added explanations describing:

- The risks being accepted.
- Why the reduced testing approach was appropriate.
- The conditions under which the team would revisit those decisions and increase testing coverage.

Why we made these changes:

The AI-assisted review helped the team connect testing priorities directly to business risk and provided justification for accepted testing limitations. This improved the discussion by explaining not only what the team intends to test, but also why certain areas receive greater or lesser testing attention.

Future Revisit?

Yes.

Reason:

The team will revisit these decisions if system usage patterns change, if account-management or employee-profile defects become more common, or if operational experience shows that the currently accepted risks are producing business impacts greater than originally anticipated. Additional testing effort may also be required if new functionality is added to rental processing, account management, or employee maintenance workflows.

AI Use Owner: Angela Colton

A.I. Use Record – Development of the Software Quality and Test Plan

The Component: Week 7 – Software Quality and Test Plan

Which AI Tool Was Used: ChatGPT

What Was Asked:

The Week 2 Project Charter, Week 3 Requirements Document, and Week 7 lecture transcript were provided to ChatGPT. The AI was asked to:

"Develop an initial Software Quality and Test Plan for the Golden Bike Shop using our established requirements and selected development lifecycle. Include quality assurance practices, testing responsibilities, escaped defect remediation, five detailed test cases mapped to acceptance criteria, and testing limitations and accepted risks."

What It Returned:

ChatGPT generated an initial draft of the document containing the required sections. It also provided five detailed testing scenarios covering product inventory searches, rental bike availability, service appointment conflicts, inventory synchronization, and role-based access control. Each test included preconditions, execution steps, expected results, and failure criteria.

What Was Changed and Why:

The AI-generated content was used as a starting point rather than a finalized document. I organized the draft around the assignment requirements and focused on maintaining consistency with the team's previously established decisions.

Particular attention was given to connecting the testing strategy to our evolutionary lifecycle with Agile practices and ensuring that the five test cases reflected actual acceptance criteria from the requirements document.

I also refined the named ownership description to clarify my responsibility for creating and organizing the initial document.

Why These Changes Were Made:

These refinements were intended to keep the test plan aligned with the Golden Bike Shop's business requirements while avoiding unnecessary functionality or testing expectations outside the project's scope. AI was used to accelerate the initial drafting process, while the proposed procedures and testing decisions remain subject to team review.

Future Revisit? Yes.

Reason:

The team may need to revisit the test plan as implementation progresses, especially if requirements change, new integration problems are discovered, or stakeholder feedback identifies additional testing needs.

AI Use Owner: Joseph Frongillo

Week 7 – Software Quality & Test Plan

Week ending Oct. 11, 2026

Team Decision Record

Project: The Golden Bike Shop

Team: Kickstarters

Week Ending: October 18, 2026

Deliverable: Software Quality and Test Plan

Disagreements

- ☐ No significant disagreements occurred.
- ☒ One or more disagreements occurred.

Team Decision Record

Project: The Golden Bike Shop

Team: Kickstarters

Week Ending: October 18, 2026

Deliverable: Final Design Package & Presentation (reconciliation of the six components)

Disagreements

- ☐ No significant disagreements occurred.
- ☒ One or more disagreements occurred.

The decisions below cover how the team reconciled the six earlier components into one consistent package. Each one corresponds to an entry in Section 08, Discrepancy and Resolution Notice.

Decision 1 — Where to Document Reconciliation Changes

Decision on the issue: Corrections to earlier components had been recorded in several places, including a "Requirements reconciliation" section before the UI design and an "editorial control" section in an earlier draft. The team needed to decide where these belonged in the final package.

Competing Positions

Position A: List the reconciliation notes after each component they affect, so readers see each correction in context.

Position B: Compile every reconciliation item into one section at the end of the package, labeled with the component each item affects.

Resolution: We chose Position B. All reconciliation items were moved into Section 08, Discrepancy and Resolution Notice, and the duplicate editorial section was removed. Each entry names its source component in parentheses after the title so readers can trace it.

What we gave up

- Nothing given up with this decision.

Future Revisit? No. One consolidated section is easier for reviewers to check against the rubric.

Decision 2 — Adding a Reservation Requirement (REQ-014)

Decision on the issue: Future reservations were added as a feature during the Week 6 data design and appear in the ERD, but no functional requirement described them.

Competing Positions

Position A: Leave reservations as a data-design detail without a matching requirement.

Position B: Add REQ-014 so the reservation feature has a clear description and testable acceptance criteria.

Resolution: We chose Position B. The team agreed that because reservations were designed into the data model and ERD, the commitment needed to be captured as a requirement. REQ-014 now defines reservation requests, conflict rejection, cancellation, and a recheck of bike status at checkout.

What we gave up

- Some additional acceptance criteria to test.

Future Revisit? Only if the business later wants group or multi-bike reservations.

Decision 3 — Scope of NFR-001

Decision on the issue: NFR-001 covered only inventory changes, but the UI screens already relied on it for rental bike status. Aela proposed applying the 2-second target to all database interactions started from the UI.

Competing Positions

Position A: Keep NFR-001 limited to inventory updates, as written in Week 3.

Position B: Apply the 2-second SLA to every database interaction initiated from the UI.

Resolution: We chose a middle position. NFR-001 now covers operational data (inventory, rental bike status, reservations and service appointments), which must be visible to all users within 2 seconds. Reports and historical queries are excluded because long-range aggregate queries may take longer, and they remain covered by NFR-007's response-time alerting.

What we gave up

- A single, simple rule covering every interaction.
- A requirement limited to the one area where we first measured it.

Future Revisit? If reporting performance becomes a stakeholder concern, a separate reporting response-time requirement could be added.

Decision 4 — Service Appointments for Rental Bikes

Decision on the issue: The ERD linked ServiceAppointment only to customer bikes and required a non-null customer bike ID, while the requirements and architecture say rental bikes are serviced too.

Competing Positions

Position A: Create a separate appointment table for rental bikes.

Position B: Keep one ServiceAppointment table and allow it to link to either a customer bike or a rental bike.

Resolution: We chose Position B. Each appointment now links to exactly one bike, either a customer bike or a rental bike, so conflict checking stays in one place. The ERD was updated to match.

What we gave up

- The simplicity of a single required foreign key on every appointment.

Future Revisit? No.

Decision 5 — Telemetry Storage and Detail

Decision on the issue: The Week 5 architecture only retrieved telemetry from the external system, while the Week 6 data design added a TelemetryReport table. Earlier documents also described more detailed telemetry (battery voltage, estimated range, lock settings) than the final data model stores.

Competing Positions

Position A: Retrieve telemetry on demand only, and keep the detailed fields listed in REQ-012.

Position B: Persist telemetry reports, and simplify the stored data to location, battery level and mileage.

Resolution: We chose Position B. Storing reports lets the system flag unhealthy bikes and keep working during a telemetry outage. REQ-012 and the Telemetry Integration Service were updated to match the simplified data.

What we gave up

- Detailed battery-voltage and lock-setting monitoring inside our system.

Future Revisit? Yes, if the shop wants deeper battery-health monitoring later.

Decision 6 — Independent Releases of System Tiers

Decision on the issue: The first draft of the Deployment and Maintenance Outlook implied every release redeploys the whole system. A team member asked whether a UI-only or database-only change, such as adding an index, could be released on its own.

Competing Positions

Position A: Release all tiers together in every release.

Position B: Version and release each tier (UI, backend, database) independently, with the backend as one deployable unit.

Resolution: We chose Position B. Each release is recorded as a baseline listing the version of every tier, testing is scaled to the type of change, and a full regression suite runs for backend releases.

What we gave up

- The simplicity of a single release process for every change.

Future Revisit? No.

Decision 7 — Starting a New Consolidated Document

Decision on the issue: Copying the weekly documents into one file created repeated formatting and alignment problems.

Competing Positions

Position A: Keep correcting formatting in the existing combined draft.

Position B: Start a new consolidated document (MSSE601_GoldenBikeShopPresentation_Kickstarters.docx) with consistent formatting and a linked table of contents, then move content into it.

Resolution: We chose Position B so the team could spend its time on missing content instead of formatting fixes.

What we gave up

- Some rework to reformat pasted content into the new file.

Future Revisit? No.

AI Use Record

AI Use Record – Presentation, Speaker Notes and Presenter Script

The Component: Week 8 – Final Design Package & Presentation

Which AI Tool Was Used: Claude (Anthropic)

What was asked: The six weekly components (Project Charter, System Requirements, UI Design, Application Architecture with the ERD, Data Design, and Software Quality & Test Plan) and the Week 8 syllabus requirements were provided. Claude was asked to create a PowerPoint presentation for a 10-minute recorded presentation, then to write speaker notes addressed to stakeholders that defend the team's design rather than describe each slide, and a separate printed presenter script.

What it returned:

- A slide deck with one slide per component, plus slides on inconsistencies, deployment and operations, and two hard stakeholder questions with answers.
- Speaker notes and a printed script timed to about 10 minutes.

What we changed and why:

- Reordered the slides to follow the order the components were created.
- Removed the overview, inconsistencies, hard-question and closing slides that the team decided were not pertinent to the recorded presentation.
- Removed references to AI from the data design slide.
- Asked for an added slide showing the data model in a readable form.
- Removed the time reserved for questions, since the presentation is recorded, and asked for the script to be extended to fill about 10 minutes.

Why We Made These Changes: AI was used to speed up drafting, but the team decided which slides fit our message and how the presentation should be framed.

AI Use Owner: Angela Colton

Named Ownership

Week 8 – Final Design Package Owner: Aela Mason

Week 8 – Presentation Owner: Angela Colton

Aela Mason and Angela Colton shared the responsibility for assembling the six components into one package and ensuring inconsistencies were resolved. Aela Mason created the section that identifies all the changes from prior weeks and made sure the documents from the previous weeks reflected those changes. Angela Colton was responsible for creating and recording the presentation that defends the team's significant decisions.

Individual Contribution Log

Team Member	Contribution
Aela	Owned the Final Design Package. Wrote the Discrepancy and Resolution Notice. Raised the NFR-001 scope question and the issue with independent tier releases. Identified that ServiceAppointment did not support rental bikes in the ERD. Fixed formatting in the screens section.
Angela	Owned the presentation and presenter script. Created the new consolidated document with consistent formatting. Revised the Release and Rollback section. Renumbered NFR-007 to NFR-006 throughout the package. Added component labels to each entry in the Discrepancy and Resolution Notice. Recorded the AI use for the presentation and supporting text. Created the Team Decision records.
Joseph	Had AI generate a PDF that had to be discarded. Aela was able to use the generated discrepancies listed as a starting off point for the Word document that allowed Angela and Aela to finish editing the document.